The Journal of Open Source Software

TransiScope: An Interactive Platform for Automated Detection and Analysis of Transient Events in Time-Lapse Microscopy

Rinki Dasgupta [1,2] **and Kaushik Das** [3¶]

1 Department of Psychiatry and Neuroscience, Dell Medical Center, University of Texas at Austin, Austin, TX, 78701, USA **2** Department of Biology, Texas Woman's University, Denton, TX, 76204, USA **3** Department of Computer Applications, National Institute of Technology Jharkhand, Jamshedpur, 831014, India ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 30 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Quantifying transient cellular events—such as calcium transients, vesicle exocytosis, and fluorescence fluctuations—is fundamental to understanding biological processes at the cellular level. `TransiScope` is an open-source, user-friendly desktop application built on Python and the Napari viewer that streamlines the complete workflow of detecting and analyzing such events in time-lapse microscopy data. The software provides an intuitive graphical interface that eliminates the need for programming expertise, while offering advanced features including interactive region-of-interest (ROI) management, data-driven parameter optimization, and multiple event detection algorithms (Difference of Gaussians, Otsu thresholding, step detection). Key innovations include automated parameter estimation from user-selected ROIs, complete algorithmic transparency through automated logging, and normalized quantification (events/sec/μm^2) that enables direct comparison across experiments. TransiScope fills a critical usability gap in bioimage analysis, making sophisticated temporal event detection accessible to biologists without computational backgrounds.

Statement of Need

Analyzing dynamic cellular processes from time-lapse microscopy requires detecting and quantifying transient events—discrete, short-lived changes in fluorescence intensity that represent biological phenomena such as neurotransmitter release, calcium signaling, or vesicle fusion (Hummer et al., 2017; Pelet et al., 2012). While powerful algorithms for event detection exist, including Difference of Gaussians (DoG) filtering (Yuan et al., 2014), Otsu thresholding (Cao et al., 2019), and peak detection methods, applying these techniques currently requires either significant programming expertise or navigating complex, multi-step workflows that introduce subjective bias through manual parameter tuning (O'Brien, 2016; Savage et al., 2023).

Existing bioimage analysis platforms face specific limitations for temporal event analysis:

- **ImageJ/Fiji** (Schindelin et al., 2012) with plugins like TrackMate (Cayuela López & Moriel-Carretero, 2023) excel at particle tracking and segmentation but require complex, multi-step workflows for temporal event detection within user-defined regions
- **CellProfiler** (Carpenter et al., 2006) is optimized for high-throughput segmentation-based analysis but not for analyzing rapid intensity-based temporal events
- **Specialized tools** like ThunderSTORM (Ovesný et al., 2014) focus on single-molecule localization microscopy, not functional temporal dynamics

This landscape reveals a critical usability gap: **no existing tool provides a unified, user-friendly workflow for biologists to perform robust temporal event analysis without coding skills.**

`TransiScope` addresses this gap through four key innovations:

1. **Interactive, data-driven parameter optimization**: Users draw multiple ROIs over representative cellular regions, and the software automatically analyzes their averaged intensity traces to propose optimal detection parameters. This eliminates subjective manual tuning while adapting to the specific signal characteristics of each dataset (Held et al., 2013).
2. **Unified workflow in a single GUI**: Complete analysis from file import (AVI, TIFF) through ROI drawing, event detection, and results export occurs within one intuitive interface built on Napari (Chiu et al., 2022)—no scripting, macro programming, or file format conversions required.
3. **Automated normalization for cross-experiment comparison**: Event rates are automatically normalized by both time and ROI area (events/sec/μm^2), accounting for variations in acquisition duration and region size that would otherwise confound biological interpretation.
4. **Complete algorithmic transparency and reproducibility**: All parameters, detection thresholds, and processing steps are automatically logged with timestamps, ensuring that analyses are fully reproducible and scientifically auditable (Dudda & others, 2025).

By making sophisticated event detection accessible while maintaining scientific rigor, `TransiScope` empowers a broader community of researchers to perform quantitative temporal analysis of cellular dynamics. The software is particularly valuable for laboratories studying neurotransmission, calcium signaling, exocytosis, and other rapid cellular processes where manual analysis is prohibitively time-consuming and prone to observer bias.

Software Description

Architecture

`TransiScope` follows a modular architecture with three logical layers:

- **User Interface Layer**: Built on Napari, providing interactive image display, ROI drawing tools (rectangle, ellipse, polygon, freehand), and a comprehensive control panel for parameter configuration
- **Application Logic Layer**: Pure Python modules handling file I/O (`io_operations.py`), ROI management and intensity extraction (`roi_handler.py`), event detection algorithms (`analysis_processor.py`), and logging (`utilities.py`)
- **External Libraries**: Leverages established open-source tools including NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), scikit-image (Van Der Walt et al., 2014), OpenCV (Bradski, 2000), and Shapely for computational tasks

Key Features

Data Import and Preprocessing

- Supports `.avi` video files and multi-page `.tif`/`.tiff` image stacks
- Automatic conversion of color videos to greyscale
- Microscope calibration input (pixel size in μm) for physical unit calculations

Interactive ROI Management

- Multiple drawing modes (rectangle, ellipse, polygon, freehand path) via Napari's shapes layer
- Real-time visualization of ROIs overlaid on time-lapse data
- ROI area calculation using Shapely for accurate normalization

DRAFT

Event Detection Algorithms

Difference of Gaussians (DoG): Detects blob-like features by subtracting two Gaussian-blurred versions of the intensity trace, with parameters σ (narrow filter), σ (broad filter), and minimum prominence threshold. The algorithm identifies local maxima in the DoG-filtered signal that exceed the prominence criterion.

Threshold-based Detection: Identifies events when intensity exceeds a threshold, using either manual thresholds or automatic Otsu's method (Jumiawi & El-Zaart, 2022) that analyzes the intensity distribution to establish an objective cutoff.

Step Detection (Scisson-like): Under development; intended for analyzing step-like intensity changes using change-point detection methods from the Ruptures library.

Automated Parameter Optimization

The "Auto-set Parameters" feature implements a data-driven workflow:

1. User draws multiple ROIs over representative active regions
2. Software calculates the mean intensity trace across all ROIs
3. Peak detection (via `scipy.signal.find_peaks`) identifies the most prominent peak
4. Peak width at half-maximum (FWHM) is converted to optimal σ = FWHM / 2.355
5. Broader kernel set to $\sigma = 1.6 \times \sigma$ following Marr-Hildreth convention
6. Prominence threshold set to 25% of maximum peak prominence

All calculations are logged, ensuring full transparency and reproducibility.

Post-Processing and Normalization

- **Temporal filtering**: Consolidates closely-spaced detections using a minimum separation parameter (e.g., 1-4 seconds), preventing double-counting of single biological events
- **Normalization**: Final event counts divided by video duration (seconds) and ROI area (μm^2) to yield standardized rates for cross-experiment comparison

Results Export

- Detailed CSV output with event timing, type, ROI ID, and normalized rates
- Metadata headers include analysis timestamp and parameters
- UTF-8-SIG encoding ensures proper rendering of special characters (µm)

Quality Assurance

The software has been validated using publicly available microscopy datasets (Hummer et al., 2017, 2020), demonstrating:

- **High specificity**: Background ROIs correctly identify zero events (no false positives)
- **Biological sensitivity**: High-K stimulation datasets show expected 2.5-3× increase in event rates compared to unstimulated conditions
- **Objectivity**: Automated Otsu thresholding eliminates false positives that arise with manual thresholding
- **Parameter robustness**: Systematic variation of temporal filtering parameters (min_sep) shows expected and interpretable effects on event counts

Example Usage

A typical workflow proceeds as follows:

1. **Launch application**: `transiscope` from command line
2. **Load data**: Click "Load File" and select `.avi` or `.tiff` file

DRAFT

3. **Set calibration**: Enter pixel size (e.g., 0.16 μm/pixel) for accurate area calculations
4. **Draw ROIs**: Select drawing mode and outline cellular regions of interest
5. **Optimize parameters**: Click "Auto-set Parameters" to derive optimal settings from ROI intensity traces
6. **Run analysis**: Click "Run Analysis" to detect events across all ROIs
7. **Review results**: Examine results table showing event timing and normalized rates
8. **Export**: Save results to CSV for statistical analysis or plotting

A comprehensive Jupyter notebook tutorial is included in the `examples/` directory, providing step-by-step guidance with sample data.

Acknowledgements

We thank the authors of the publicly available microscopy datasets used for validation (Hummer et al., 2017, 2020). This work leverages the open-source scientific Python ecosystem, particularly Napari, NumPy, SciPy, and scikit-image. Development was conducted as an independent open-source initiative without dedicated institutional funding.

References

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*, *25*.

Cao, X., Ding, M., Su, Y., Guo, Y., Wang, H., & Jia, W. (2019). A robust parameter-free thresholding method for image segmentation. *IEEE Access*, *7*, 3448–3458. https://doi.org/10.1109/ACCESS.2018.2889013

Carpenter, A. E., Jones, T. R., Lamprecht, M. R., Clarke, C., Kang, I. H., Friman, O., Guertin, D. A., Chang, J. H., Lindquist, R. A., Moffat, J., Golland, P., & Sabatini, D. M. (2006). CellProfiler: Image analysis software for identifying and quantifying cell phenotypes. *Genome Biology*, *7*. https://doi.org/10.1186/gb-2006-7-10-r100

Cayuela López, A., & Moriel-Carretero, M. (2023). TrackAnalyzer: A Fiji/ImageJ toolbox for a holistic analysis of tracks. *Biological Imaging*, *3*. https://doi.org/10.1017/S2633903X23000181

Chiu, C.-L., Clack, N., & the napari community. (2022). Napari: A Python multi-dimensional image viewer platform for the research community. *Microscopy and Microanalysis*, *28*, 1576–1577. https://doi.org/10.1017/S1431927622006328

Dudda, L., & others. (2025). Open science interventions to improve reproducibility and replicability of research: A scoping review. *Royal Society Open Science*, *12*. https://doi.org/10.1098/rsos.240457

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Held, C., Nattkemper, T., Palmisano, R., & Wittenberg, T. (2013). Approaches to automatic parameter fitting in a microscopy image segmentation pipeline: An exploratory parameter space analysis. *Journal of Pathology Informatics*, *4*, 5. https://doi.org/10.4103/2153-3539.109868

Hummer, B. H., Leeuw, N. F. de, Burns, C., Chen, L., Joens, M. S., Hosford, B., Fitzpatrick, J. A. J., & Asensio, C. S. (2017). HID-1 controls formation of large dense core vesicles by influencing cargo sorting and trans-Golgi network acidification. *Molecular Biology of the Cell*, *28*, 3870–3880. https://doi.org/10.1091/mbc.E17-08-0491

DRAFT

Hummer, B. H., Maslar, D., Soltero-Gutierrez, M., Leeuw, N. F. de, & Asensio, C. S. (2020). Differential sorting behavior for soluble and transmembrane cargoes at the trans-Golgi network in endocrine cells. *Molecular Biology of the Cell*, *31*, 157–166. https://doi.org/10.1091/mbc.E19-08-0429

Jumiawi, W. A. H., & El-Zaart, A. (2022). Improvement in the Between-Class variance based on lognormal distribution for accurate image segmentation. *Entropy*, *24*. https://doi.org/10.3390/e24091274

O'Brien, J. (2016). Automated quantification and analysis of cell counting procedures using ImageJ plugins. *Journal of Visualized Experiments*, *117*. https://doi.org/10.3791/54719

Ovesný, M., Křížek, P., Borkovec, J., Švindrych, Z., & Hagen, G. M. (2014). ThunderSTORM: A comprehensive ImageJ plug-in for PALM and STORM data analysis and super-resolution imaging. *Bioinformatics*, *30*, 2389–2390. https://doi.org/10.1093/bioinformatics/btu202

Pelet, S., Dechant, R., Lee, S. S., Drogen, F. van, & Peter, M. (2012). An integrated image analysis platform to quantify signal transduction in single cells. *Integrative Biology*, *4*, 1274–1282. https://doi.org/10.1039/c2ib20139a

Savage, J. T., Ramirez, J., Risher, W. C., Irala, D., & Eroglu, C. (2023). SynBot: An open-source image analysis software for automated quantification of synapses. *bioRxiv*. https://doi.org/10.1101/2023.06.26.546578

Schindelin, J., Arganda-Carreras, I., Frise, E., Kaynig, V., Longair, M., Pietzsch, T., Preibisch, S., Rueden, C., Saalfeld, S., Schmid, B., Tinevez, J.-Y., White, D. J., Hartenstein, V., Eliceiri, K., Tomancak, P., & Cardona, A. (2012). Fiji: An open-source platform for biological-image analysis. *Nature Methods*, *9*, 676–682. https://doi.org/10.1038/nmeth.2019

Van Der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., & Yu, T. (2014). Scikit-image: Image processing in Python. *PeerJ*. https://doi.org/10.7717/peerj.453

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Yuan, Y., Liu, Y., Dai, G., Zhang, J., & Chen, Z. (2014). Automatic foreground extraction based on difference of Gaussian. *Scientific World Journal*. https://doi.org/10.1155/2014/208073

DRAFT